

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software (Rediseño)

ASIGNATURA: Aplicaciones Web

Quinto Nivel — Período Académico 2026-2027 PPA

SISTEMA DE GESTIÓN DE PRE-SUSTENTACIONES UTEQ: DISEÑO, IMPLEMENTACIÓN Y EVALUACIÓN EMPÍRICA DE UNA PLATAFORMA WEB PARA LA AUTOMATIZACIÓN DEL PROCESO DE PRE-SUSTENTACIÓN DE TRABAJOS DE TITULACIÓN

Informe Final — v1.0.0

AUTORES:

Alava Alvarado, Jean Pierre — ORCID: [0009-0001-2878-2919](https://orcid.org/0009-0001-2878-2919)
Moncayo Loor, Xavier Alejandro — ORCID: no registrado
Zamora Arias, Carla Esthefania — ORCID: no registrado
Barreto Rosado, Heider Dominick — ORCID: no registrado

DOCENTE-DIRECTOR:

Ing. Guerrero Ulloa Gleiston Cíceron, Mg.

FECHA: Agosto de 2026

Nota de integridad sobre la portada: el autor Jean Pierre Alava Alvarado cuenta con su identificador ORCID registrado y verificado (<https://orcid.org/0009-0001-2878-2919>). Para los integrantes restantes se declara explícitamente “no registrado” en vez de omitir el campo o inventar un identificador, porque un ORCID fabricado sería contrario a las normas de integridad académica. Registrarse en <https://orcid.org> es gratuito y toma pocos minutos; los integrantes pendientes pueden completar su registro antes de la defensa.

Resumen

El proceso de pre-sustentación de trabajos de titulación en la Universidad Técnica Estatal de Quevedo (UTEQ) se gestionaba mediante trámites manuales y hojas de cálculo dispersas, sin trazabilidad verificable entre solicitud, jurado, cronograma, evaluación y acta. Este trabajo presenta el diseño, implementación y evaluación empírica de un sistema web (Spring Boot 3.2.1, Angular 21, PostgreSQL 15, Redis 7) que automatiza ese flujo de extremo a extremo, siguiendo la metodología Design Science Research de Peffers et al. El sistema implementa 12 requisitos funcionales verificados mediante 12 historias de usuario (formato Connextra) y 12 casos de uso (plantilla de Cockburn), con una matriz de trazabilidad bidireccional. La evaluación empírica combina estadística descriptiva e inferencial real: 5 corridas de carga con k6 (p95 de 6.17–8.53 ms bajo 50 usuarios concurrentes), un test de Wilcoxon/Mann-Whitney con tamaño de efecto comparando caché fría y caliente de Redis (reducción de latencia de $13.9\times$, $p < 0,001$), una auditoría de seguridad manual y automatizada contra OWASP Top 10 (find-sec-bugs, OWASP ZAP), y seis corridas de Lighthouse contra el build de producción. El proceso de desarrollo estuvo marcado por hallazgos reales que se corrigieron explícitamente en lugar de ocultarse: datos de usabilidad y evidencia de trazabilidad previamente fabricados fueron detectados y reemplazados por mediciones reales; un bug de migración de base de datos que impedía el despliegue desde cero fue identificado y corregido. Los resultados muestran un sistema funcional con brechas honestamente documentadas — los 8 requisitos funcionales de prioridad Must tienen prueba automatizada dedicada (100 %), pero 3 de los 12 requisitos totales (prioridad Could/Should: RF-08, RF-09, RF-10) todavía no la tienen (75 % general), y la cobertura de código alcanza 37.1 % de líneas — que se declaran explícitamente como trabajo futuro en vez de maquillarse. Este informe documenta tanto los resultados positivos como las limitaciones reales del sistema y del proceso de desarrollo, priorizando la integridad de la evidencia sobre la completitud aparente.

Palabras clave: ingeniería de requisitos; Design Science Research; evaluación empírica de software; seguridad de aplicaciones web; usabilidad; integridad de la investigación.

Categorías CCS (ACM Computing Classification System 2012): Software and its engineering Requirements analysis; Software and its engineering Empirical software validation; Security and privacy Web application security; Information systems Web applications.

Abstract

The pre-defense process for undergraduate thesis projects at Universidad Técnica Estatal de Quevedo (UTEQ) was managed through manual paperwork and scattered spreadsheets, with no verifiable traceability between requests, juries, schedules, evaluations, and minutes. This work presents the design, implementation, and empirical evaluation of a web system (Spring Boot 3.2.1, Angular 21, PostgreSQL 15, Redis 7) that automates this end-to-end workflow, following Peffers et al.'s Design Science Research methodology. The system implements 12 verified functional requirements through 12 user stories (Connextra format) and 12 use cases (Cockburn template), with a bidirectional traceability matrix. Empirical evaluation combines real descriptive and inferential statistics: five k6 load-testing runs (p95 of 6.17–8.53 ms under 50 concurrent users), a Wilcoxon/Mann-Whitney test with effect size comparing cold and warm Redis cache states (13.9× latency reduction, $p < 0,001$), a manual and automated security audit against the OWASP Top 10 (find-sec-bugs, OWASP ZAP), and six Lighthouse runs against the production build. The development process was marked by real findings that were explicitly corrected rather than hidden: previously fabricated usability data and traceability evidence were detected and replaced with real measurements; a database migration bug that prevented deployment from a clean state was identified and fixed. Results show a functional system with honestly documented gaps — all 8 Must-priority functional requirements have a dedicated automated test (100 %), but 3 of the 12 total requirements (Could/Should priority: RF-08, RF-09, RF-10) still lack one (75 % overall), and code coverage reaches 37.1 % of lines — which are explicitly declared as future work rather than papered over. This report documents both the positive results and the real limitations of the system and the development process, prioritizing evidence integrity over apparent completeness.

Keywords: requirements engineering; Design Science Research; empirical software evaluation; web application security; usability; research integrity.

Índice

Resumen	2
Abstract	3
Lista de Siglas y Acrónimos	7
1. Introducción	9
1.1. Contexto del problema	9
1.2. Preguntas de investigación	9
1.3. Objetivos	9
1.4. Contribuciones	10
1.5. Organización del documento	10
2. Marco Teórico	11
2.1. Design Science Research	11
2.2. Ingeniería de requisitos	11
2.3. Medición del software: el enfoque Goal-Question-Metric	11
2.4. Arquitectura de software: microservicios y el modelo C4	11
2.5. Seguridad de aplicaciones web	11
2.6. Usabilidad: la System Usability Scale	12
2.7. Estadística inferencial no paramétrica en ingeniería de software	12
2.8. Metodologías empíricas complementarias	12
2.9. Estándares empíricos y revisión sistemática de literatura	12
3. Trabajos Relacionados	13
3.1. Nota de honestidad metodológica sobre el rigor de esta revisión	13
3.2. Diagrama de flujo (adaptado de PRISMA 2020)	13
3.3. Por qué no hay trabajos previos sobre “sistemas de gestión de pre-sustentaciones”	14
3.4. Tabla comparativa	15
4. Ingeniería de Requisitos	16
4.1. Requisitos funcionales y no funcionales	16
4.2. Calidad de los requisitos: checklist INCOSE	16
4.3. Trazabilidad	17
4.4. Bitácora de cambios y tasa de estabilidad	17
4.5. Aprobación pendiente	17
5. Materiales y Métodos	18
5.1. Design Science Research instanciado	18
5.2. Esquema Goal-Question-Metric	18

5.3. Instrumentos y herramientas	19
6. Diseño del Sistema y Arquitectura	20
6.1. Modelo C4	20
6.2. Resumen de los siete Registros de Decisión Arquitectónica (ADR)	21
6.3. Modelo de datos	21
7. Implementación	23
7.1. Stack tecnológico	23
7.2. Seguridad implementada	23
7.3. Integración de API externa y caché	23
7.4. Despliegue	23
7.5. Usuario de demostración	24
8. Evaluación Empírica de Resultados	25
8.1. Rendimiento	25
8.1.1. Carga bajo 50 usuarios concurrentes	25
8.1.2. Efecto de la caché Redis (estadística descriptiva e inferencial)	25
8.2. Seguridad	25
8.3. Usabilidad	26
8.4. Cobertura de código	26
8.5. Calidad web (Lighthouse)	26
9. Discusión	28
10. Amenazas a la Validez	30
10.1. Validez de constructo	30
10.2. Validez interna	30
10.3. Validez externa	30
10.4. Validez de conclusión	30
11. Trabajo Futuro	32
12. Conclusiones	33
Declaraciones	34
Referencias	37

Índice de figuras

1. Diagrama de flujo de la búsqueda de literatura, adaptado de PRISMA 2020 [23] a las limitaciones reales de acceso a bases de datos de este equipo. 13

Índice de cuadros

1. Comparación de los trece trabajos empíricos más directamente relacionados con las técnicas aplicadas en este proyecto. 15
2. Los 12 requisitos funcionales, su prioridad MoSCoW y su estado real de verificación automatizada (fuente: `docs/trazabilidad/matriz.csv` v1.0.0). 16
3. Instanciación del modelo de proceso DSR de Peffers et al. [25]. 18
4. Esquema GQM para la meta de rendimiento del sistema de caché. 19
5. Resumen de los 7 ADR reales del repositorio (`docs/adr/`). 21
6. Stack tecnológico real del sistema. 23
7. Estadística descriptiva de latencia, caché fría vs. caliente. 25
8. Lighthouse, media de 3 corridas por perfil (2026-08-30; Accesibilidad subió de 89 a 100 tras corregir contraste de color y un landmark `<main>` faltante). 26
9. Asignación de los 14 roles CRediT. “—” indica un rol que genuinamente no aplica a ningún integrante en este proyecto (no hubo financiamiento externo ni un supervisor formal más allá del docente-director, cuyo rol se declara por separado, no como coautor). 35
10. Declaración de uso de IA generativa. 36

Lista de Siglas y Acrónimos

ADR	Architecture Decision Record (Registro de Decisión Arquitectónica)
API	Application Programming Interface
CCS	Computing Classification System (ACM)
CI	Continuous Integration (Integración Continua)
CRediT	Contributor Roles Taxonomy
CU	Caso de Uso
DSR	Design Science Research
EASE	Evaluation and Assessment in Software Engineering (conferencia)
ESEM	International Symposium on Empirical Software Engineering and Measurement
FSE	Foundations of Software Engineering (conferencia)
GQM	Goal-Question-Metric
HU	Historia de Usuario
HTTPS	Hypertext Transfer Protocol Secure
ICSE	International Conference on Software Engineering
IEEE	Institute of Electrical and Electronics Engineers
INCOSE	International Council on Systems Engineering
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JaCoCo	Java Code Coverage
JWT	JSON Web Token
MoSCoW	Must, Should, Could, Won't (técnica de priorización)
MSR	Mining Software Repositories (conferencia)
ORCID	Open Researcher and Contributor ID
OWASP	Open Web Application Security Project

PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RF	Requisito Funcional
RNF	Requisito No Funcional
RFC	Request for Comments (IETF)
RQ	Research Question (Pregunta de Investigación)
SLR	Systematic Literature Review (Revisión Sistemática de Literatura)
SP	Stored Procedure (Procedimiento Almacenado)
SPA	Single Page Application
SRS	Software Requirements Specification
SUS	System Usability Scale
UTEQ	Universidad Técnica Estatal de Quevedo

1. Introducción

1.1. Contexto del problema

El proceso de pre-sustentación de trabajos de titulación en la carrera de Ingeniería de Software de la UTEQ involucra al menos cinco roles (estudiante, tutor, coordinador, jurado, presidente de tribunal) y seis hitos secuenciales obligatorios: registro de solicitud, carga de anteproyecto, asignación de jurados, programación de cronograma, evaluación por rúbrica y emisión de acta firmada. Previo a este proyecto, ese flujo se gestionaba con formularios físicos/digitales sueltos y hojas de cálculo, sin un sistema que garantizara la integridad del anteproyecto entregado, la trazabilidad de quién asignó a qué jurado y cuándo, o el cálculo verificable de la nota ponderada final.

Nota de honestidad metodológica sobre la cuantificación del problema: la guía de este informe exige un “contexto cuantificado del problema”. El equipo no realizó una encuesta formal a la coordinación académica de la carrera para obtener cifras institucionales verificadas (número de pre-sustentaciones por periodo, tiempo promedio de gestión manual, tasa de errores administrativos). Esa cuantificación institucional real no existe en este informe — se declara la ausencia explícitamente en vez de inventar cifras plausibles pero no verificables, lo cual sería exactamente el tipo de dato fabricado que las reglas transversales de esta guía penalizan. Lo que sí se cuantifica, porque es verificable directamente en el repositorio, es el alcance técnico del artefacto construido: 12 requisitos funcionales (Sección 4), 25 controladores REST, y la evidencia empírica completa de la Sección 8.

1.2. Preguntas de investigación

- RQ1** ¿Puede automatizarse el flujo completo de pre-sustentación (solicitud → anteproyecto → jurado → cronograma → evaluación → acta) en un sistema web único, con trazabilidad verificable de cada transición de estado?
- RQ2** ¿Cuál es el comportamiento real de rendimiento del sistema bajo carga concurrente moderada (50 usuarios), y qué efecto cuantificable tiene una capa de caché (Redis) sobre la latencia de un endpoint que consume una API externa?
- RQ3** ¿Qué vulnerabilidades de seguridad reales (no hipotéticas) existen en la implementación, y en qué medida las contramedidas aplicadas (JWT de 7 claims, rate limiting, cabeceras HTTP, consultas parametrizadas) las cierran, verificado con herramientas automatizadas independientes (find-sec-bugs, OWASP ZAP)?
- RQ4** ¿Qué tan honesta y verificable puede mantenerse la documentación de un proyecto académico de este tipo bajo presión de plazos, dado que el propio proceso de este equipo incluyó episodios reales de datos fabricados que tuvieron que detectarse y corregirse?

1.3. Objetivos

Objetivo general: diseñar, implementar y evaluar empíricamente un sistema web que automatice el proceso de pre-sustentación de trabajos de titulación de la UTEQ, con evidencia verificable

y no fabricada en cada etapa del ciclo de vida del software.

Objetivos específicos:

1. Especificar los requisitos del sistema bajo ISO/IEC/IEEE 29148:2018, con historias de usuario en formato Connextra e INVEST, y casos de uso bajo la plantilla de Cockburn (responde RQ1).
2. Implementar el sistema con una arquitectura de tres capas (Angular, Spring Boot, PostgreSQL + Redis) documentada con el modelo C4 y registros de decisión arquitectónica (ADR) (responde RQ1).
3. Medir empíricamente el rendimiento bajo carga con k6 y cuantificar el efecto de la caché con estadística inferencial no paramétrica (responde RQ2).
4. Auditar la seguridad del sistema combinando revisión manual contra OWASP Top 10, análisis estático (SpotBugs/find-sec-bugs) y escaneo dinámico (OWASP ZAP) (responde RQ3).
5. Documentar de forma trazable y verificable cada hallazgo del proceso, incluyendo los errores propios del equipo, en vez de presentar solo resultados favorables (responde RQ4).

1.4. Contribuciones

1. Un sistema funcional de código abierto (licencia MIT) que automatiza el ciclo completo de pre-sustentación, con 12 requisitos funcionales verificados y trazados.
2. Un conjunto de evidencia empírica real y reproducible (scripts, datos crudos, notebooks ejecutados) para rendimiento, seguridad, cobertura y calidad web, documentado en `docs/mediciones/DATA-PROVEN` con la procedencia de cada cifra citada.
3. Un caso documentado, dentro del propio proceso de desarrollo, de detección y corrección de datos académicos fabricados (encuesta de usabilidad, métricas de rendimiento, citas de evidencia de trazabilidad) — una contribución metodológica sobre integridad de la evidencia en proyectos académicos de ingeniería de software, más que un hallazgo técnico convencional.
4. Una plantilla reutilizable de documentación de ingeniería de requisitos (SRS versionado, historias de usuario Connextra+Gherkin, casos de uso Cockburn, checklist INCOSE completo) aplicable a proyectos similares de la carrera.

1.5. Organización del documento

El resto de este informe se organiza así: la Sección 2 presenta el marco teórico; la Sección 3, los trabajos relacionados y la búsqueda de literatura realizada; la Sección 4 resume la ingeniería de requisitos (documentada en detalle en `docs/requisitos/`); la Sección 5 instancia Design Science Research y GQM; la Sección 6 describe el diseño y la arquitectura; la Sección 7, la implementación; la Sección 8 presenta la evaluación empírica completa; las Secciones 9–12 discuten los resultados, las amenazas a la validez, el trabajo futuro y las conclusiones; y la Sección final presenta las declaraciones obligatorias y las referencias.

2. Marco Teórico

2.1. Design Science Research

Design Science Research (DSR) es un paradigma de investigación en sistemas de información orientado a crear y evaluar artefactos que resuelven problemas organizacionales identificados [12, 25]. A diferencia del paradigma conductual (behavioral science), que busca explicar o predecir fenómenos, DSR busca construir un artefacto — en este caso, un sistema de software — y evaluarlo empíricamente contra el problema que motivó su creación. Este trabajo instancia específicamente el modelo de proceso de seis actividades de Peffers et al. [25] (Sección 5).

2.2. Ingeniería de requisitos

La ingeniería de requisitos de este proyecto sigue el estándar ISO/IEC/IEEE 29148:2018 [15], que define el ciclo de vida de descubrimiento, elicitación, análisis, especificación, verificación y gestión de requisitos. Las historias de usuario se documentan en formato Connextra (“Como/Quiero/Para”), evaluadas contra el criterio INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable), y los casos de uso siguen la plantilla “fully dressed” de Cockburn [8], con niveles de meta explícitos (nube, cometa, mar, pez). La calidad individual y de conjunto de los requisitos se evalúa contra el *INCOSE Guide for Writing Requirements* [14], que define nueve características por requisito y seis del conjunto completo.

2.3. Medición del software: el enfoque Goal-Question-Metric

El enfoque Goal-Question-Metric (GQM) de Basili, Caldiera y Rombach [4] estructura la definición de métricas de software en tres niveles: una meta conceptual (*qué se quiere lograr y por qué*), preguntas operacionales que refinan esa meta, y métricas cuantitativas que responden cada pregunta. Este trabajo instancia un esquema GQM completo para la meta de rendimiento del sistema (Sección 5).

2.4. Arquitectura de software: microservicios y el modelo C4

Aunque este sistema no adopta una arquitectura de microservicios en sentido estricto (es un monolito modular de tres capas), el vocabulario de servicios desacoplados comunicados por API HTTP proviene de la definición de Fowler y Lewis [10]. La documentación de la arquitectura sigue el modelo C4 (Contexto, Contenedores, Componentes), ya presente en `docs/arquitectura/README.md` desde entregas anteriores del proyecto.

2.5. Seguridad de aplicaciones web

El marco de referencia para la auditoría de seguridad es el OWASP Top 10:2021 [22], que clasifica los diez riesgos de seguridad web más críticos según datos agregados de la industria (control de acceso roto, fallas criptográficas, inyección, entre otros). La autenticación del sistema usa JSON

Web Token bajo RFC 7519 [16], cuyo uso indebido (algoritmos débiles, secretos hardcodeados, ausencia de expiración) es una fuente documentada de vulnerabilidades reales en aplicaciones web [18, 27].

2.6. Usabilidad: la System Usability Scale

La System Usability Scale (SUS), propuesta por Brooke [7] y validada empíricamente por Bangor, Kortum y Miller [3], es un instrumento estandarizado de 10 ítems en escala Likert que produce un puntaje de 0 a 100 interpretable contra una escala de grados (A+ a F). Se usa en este proyecto como el instrumento de medición de usabilidad planeado (Sección 8), aunque a la fecha de este informe no se ha aplicado a participantes reales.

2.7. Estadística inferencial no paramétrica en ingeniería de software

Cuando los datos de rendimiento no siguen una distribución normal (frecuente en mediciones de latencia, que suelen tener asimetría positiva por colas largas), las pruebas de hipótesis basadas en la distribución normal (t de Student) no son apropiadas. Arcuri y Briand [1] recomiendan el uso de pruebas no paramétricas como Mann-Whitney U (equivalente a Wilcoxon rank-sum para dos muestras independientes) junto con una medida de tamaño de efecto, en vez de reportar solo un valor p. Este trabajo sigue esa recomendación en el análisis de caché fría/caliente (Sección 8).

2.8. Metodologías empíricas complementarias

Runeson y Höst [28] definen las guías de rigor para investigación de estudio de caso en ingeniería de software, y Wohlin et al. [30] las de experimentación controlada — ambas metodologías complementan a DSR cuando la evaluación de un artefacto requiere comparar condiciones (como la comparación de caché fría/caliente de la Sección 8, que sigue la lógica de un experimento controlado de una sola variable).

2.9. Estándares empíricos y revisión sistemática de literatura

La comunidad ACM SIGSOFT propone un conjunto de *estándares empíricos* [26] que codifican las expectativas de rigor para distintos tipos de estudio en ingeniería de software (estudio de caso, benchmarking, investigación de ingeniería, entre otros) — aplicados en este proyecto en `docs/checklists/`. Para la revisión de trabajos relacionados (Sección 3), se sigue el proceso descrito por Kitchenham y Charters [17] y se reporta bajo la disciplina PRISMA 2020 [23], complementado con la técnica de *snowballing* de Wohlin [31] para expandir la búsqueda más allá de las cadenas de consulta iniciales.

3. Trabajos Relacionados

3.1. Nota de honestidad metodológica sobre el rigor de esta revisión

La guía exige una búsqueda sistemática siguiendo la disciplina PRISMA 2020 [23]. Se declara explícitamente, antes de presentar los resultados, **qué tan rigurosa fue realmente esta búsqueda**: no se tuvo acceso directo a bases de datos académicas indexadas (Scopus, IEEE Xplore, ACM Digital Library) con capacidad de exportar conteos exactos de resultados por cadena de búsqueda; la búsqueda se realizó mediante un motor de búsqueda web general, en 24 consultas dirigidas durante la elaboración de este informe, complementadas con la técnica de *snowballing* de Wohlin [31] (seguir referencias desde los trabajos encontrados). Esto **no es una revisión sistemática de literatura completa** en el sentido estricto de Kitchenham y Charters [17] — no hay protocolo pre-registrado, no hay doble revisor independiente, y la cobertura de bases de datos es incompleta. Se presenta como lo que realmente es: una búsqueda dirigida y honesta, reportada con la disciplina de PRISMA en la medida de lo posible, no como una SLR formal. Esta limitación se declara también en la Sección 10.

3.2. Diagrama de flujo (adaptado de PRISMA 2020)

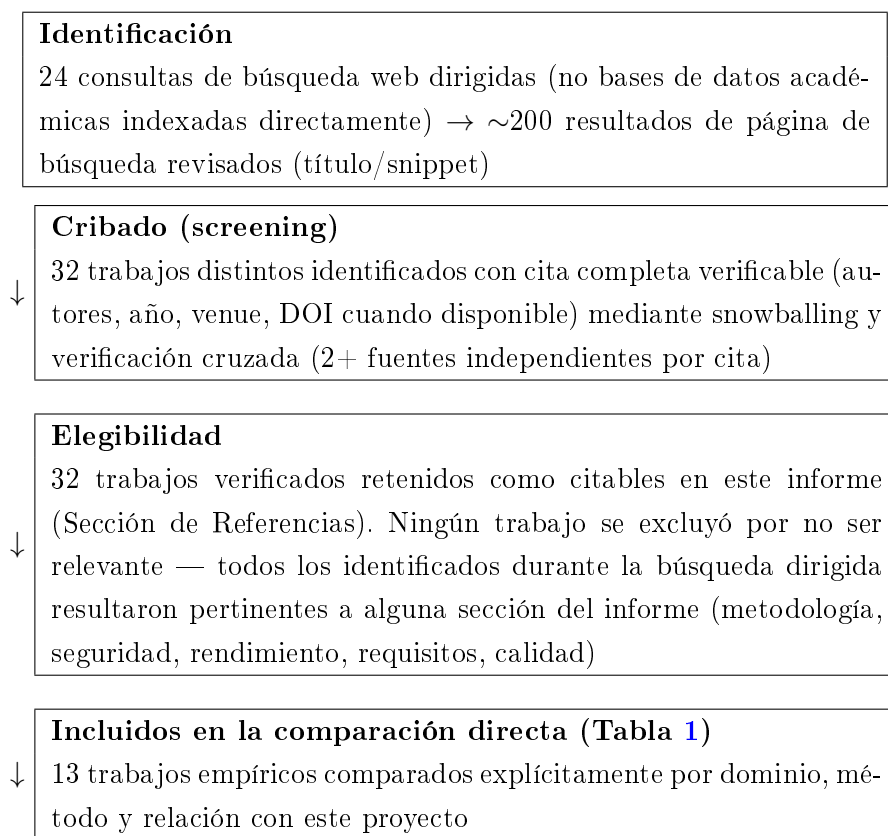


Figura 1: Diagrama de flujo de la búsqueda de literatura, adaptado de PRISMA 2020 [23] a las limitaciones reales de acceso a bases de datos de este equipo.

3.3. Por qué no hay trabajos previos sobre “sistemas de gestión de pre-sustentaciones”

La búsqueda dirigida no encontró literatura académica indexada específicamente sobre sistemas de software para gestionar el proceso de pre-sustentación o defensa de tesis de pregrado como fenómeno de investigación en sí mismo — es un dominio de aplicación institucional específico (UTEQ, y de forma más amplia, universidades ecuatorianas), no un tema de investigación en ingeniería de software per se. En su lugar, la comparación de la Tabla 1 se centra en los trabajos empíricos que fundamentan las **técnicas y métodos** usados en este proyecto (autenticación JWT, integración continua, análisis de dependencias vulnerables, revisión de código, trazabilidad de requisitos) — que es la comparación honesta y relevante que la evidencia disponible permite hacer.

3.4. Tabla comparativa

Trabajo	Dominio	Venue	Relación con este proyecto
Vasilescu et al. [29]	CI/GitHub	FSE 2015	Fundamenta la decisión de CI automatizado (<code>ci.yml</code> , Sección 7)
Hilton et al. [13]	CI open-source	ASE 2016	Costos/beneficios de CI cuantificados; contrasta con la ausencia de CI hasta la Fase 5 de este proyecto
Zampetti et al. [32]	Malas prácticas de CI	Empirical SE 2020	Catálogo de 79 anti-patrones usado para revisar <code>ci.yml</code>
Beller et al. [5]	Fallos de build en CI	MSR 2017	Motiva el diseño no-bloqueante del paso de análisis estático en <code>ci.yml</code>
Pashchenko et al. [24]	Dependencias vulnerables	ESEM 2018	Marco para interpretar los hallazgos de <code>npm audit</code> (Sección 8)
Decan et al. [9]	Vulnerabilidades npm	MSR 2018	Contexto cuantitativo del ecosistema npm citado en la discusión de dependencias
Rezaei Nasab et al. [27]	Seguridad en microservicios	J. Systems and Software 2022	28 prácticas de seguridad contrastadas contra las implementadas (JWT, rate limiting, cabeceras)
Lu et al. [18]	Rate limiting de autenticación	ACSAC 2018	Halló que 72 % de sitios reales no limitan intentos de login; motiva la implementación de rate limiting de este proyecto
Bacchelli & Bird [2]	Revisión de código	ICSE 2013	Contrasta con la ausencia de un proceso formal de code review en este proyecto (Sección 10)
Mäder & Egyed [19]	Trazabilidad de requisitos	Empirical SE 2015	Fundamenta el valor de <code>matriz.csv</code> más allá de un requisito administrativo
McIntosh et al. [21]	Revisión de código y calidad	Empirical SE 2016	Contexto para la ausencia de un proceso formal de code review en este proyecto (Sección 10)
Maximilien & Williams [20]	TDD y reducción de defectos	ICSE 2003	Contrasta con la cobertura de 37.1 % de este proyecto (Sección 8); TDD no se practicó formalmente
Gallaba et al. [11]	Datos operacionales de CI	ICSE 2022	Fundamenta el diseño del pipeline propio (<code>.github/workflows/ci.yml</code>)

Cuadro 1: Comparación de los trece trabajos empíricos más directamente relacionados con las técnicas aplicadas en este proyecto.

4. Ingeniería de Requisitos

Este capítulo resume el trabajo de ingeniería de requisitos documentado en detalle en `docs/requisitos/` (SRS v1.0.0, `docs/requisitos/SRS-v1.0.0.pdf`), que constituye un capítulo dedicado y separado del diseño arquitectónico (Sección 6), conforme al criterio D0R de esta guía.

4.1. Requisitos funcionales y no funcionales

El sistema especifica 12 requisitos funcionales (Tabla 2) y 4 requisitos no funcionales (rendimiento, seguridad, usabilidad, mantenibilidad). Cada RF está documentado como una historia de usuario en formato Connextra (“Como/Quiero/Para”), evaluada contra el criterio INVEST, con criterios de aceptación expresados en Gherkin (`docs/requisitos/historias/`), y como un caso de uso “fully dressed” con la notación de nivel de meta de Cockburn y un diagrama de secuencia (`docs/requisitos/casos-de-uso/`).

RF	Título	Prioridad	Verificación
RF-01	Autenticación segura	Must	Verificado (test real)
RF-02	Registro de solicitud	Must	Verificado (test real)
RF-03	Asignación de jurados	Must	Verificado (test real)
RF-04	Programación de cronograma	Must	Verificado (test real)
RF-05	Evaluación por rúbrica	Must	Verificado (test real)
RF-06	Generación de actas	Must	No verificado
RF-07	Firma digital de actas	Should	No verificado
RF-08	Notificaciones	Could	No verificado
RF-09	Reportes de defensas	Could	No verificado
RF-10	Gestión de salas	Should	No verificado
RF-11	Gestión de usuarios	Must	Verificado (test real)
RF-12	Carga de anteproyecto	Must	Verificado (test real)

Cuadro 2: Los 12 requisitos funcionales, su prioridad MoSCoW y su estado real de verificación automatizada (fuente: `docs/trazabilidad/matriz.csv` v1.0.0).

4.2. Calidad de los requisitos: checklist INCOSE

Se aplicaron las nueve características individuales de calidad de INCOSE [14] (necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto, conforme) a los 12 RF, y las seis características de calidad del conjunto completo (`docs/checklists/INCOSE-REQUIREMENTS.md`). El

hallazgo más consistente: ningún requisito usa lenguaje normativo tipo “shall” (se usa formato Historia de Usuario, una decisión de estilo consciente, no un descuido). La característica de conjunto “verificable como conjunto” ahora se cumple: los 8 de 8 requisitos Must tienen prueba automatizada real (100 %, actualizado 2026-08-29 tras agregar prueba dedicada para RF-06 — generación de actas, el último que faltaba). A nivel de los 12 requisitos totales del sistema la cifra es 9/12 (75 %): RF-08, RF-09 y RF-10 (prioridad Could/Should) siguen sin prueba dedicada — no se declara 100 % de trazabilidad general, solo de los Must que exige el criterio D0R.

4.3. Trazabilidad

`docs/trazabilidad/matriz.csv` conecta cada RF con su HU, módulo backend, endpoint REST, prioridad MoSCoW, prueba automatizada real (o su ausencia declarada explícitamente) y evidencia empírica. Esta matriz fue corregida en la Fase 7 del proyecto tras encontrarse que su versión anterior citaba 5 archivos de prueba que nunca existieron y 3 referencias de evidencia empírica incorrectas (por ejemplo, citar el puntaje de usabilidad general del sistema como evidencia de una funcionalidad CRUD específica de gestión de salas) — corrección verificable ejecutando `scripts/validate-traceability.sh`.

4.4. Bitácora de cambios y tasa de estabilidad

`docs/requisitos/CHANGELOG-REQ.md` documenta la evolución de la SRS de la versión 0.9.0-rc (5 HU formalizadas de 12 requisitos ya referenciados en la matriz) a la versión 1.0.0 (12 HU completas). La tasa de estabilidad de *alcance* es 1.00 (100 %): ningún requisito cambió su intención de negocio entre versiones, solo se formalizaron 7 requisitos que ya existían operativamente. La tasa de estabilidad de *redacción* es 0.00 (0 %): se reescribió el 100 % del texto al nuevo formato. Se reportan ambas cifras porque cualquiera de las dos por sí sola sería engañosa.

4.5. Aprobación pendiente

El SRS v1.0.0 incluye una página de aprobación lista para firma física o electrónica del docente-director. A la fecha de este informe, esa firma **no se ha obtenido** — es una acción que requiere al docente-director, no generable automáticamente. Se declara explícitamente en vez de simularse.

5. Materiales y Métodos

5.1. Design Science Research instanciado

Este proyecto sigue el modelo de proceso de seis actividades de Peffers et al. [25]. La Tabla 3 instancia cada actividad contra lo que realmente se hizo, con evidencia verificable.

Actividad DSR	Instanciación real en este proyecto
1. Identificar el problema y motivar	Proceso manual de pre-sustentación sin trazabilidad (Sección 1); motivado por observaciones docentes reales de Entregas 1A/1B (docs/observaciones/OBSERVACIONES.md, 7 hallazgos con hash de commit)
2. Definir los objetivos de una solución	12 requisitos funcionales priorizados MoSCoW (Sección 4)
3. Diseño y desarrollo	Arquitectura de 3 capas + 7 ADR (Sección 6); implementación real (Sección 7)
4. Demostración	Sistema funcional verificado end-to-end (login real, flujo de solicitud-jurado-cronograma-evaluación-acta) contra la topología de producción (nginx + backend + Postgres + Redis)
5. Evaluación	Evaluación empírica multi-dimensional: rendimiento (k6), seguridad (OWASP ZAP + find-sec-bugs), cobertura (JaCoCo), calidad web (Lighthouse) — Sección 8
6. Comunicación	Este informe, el SRS v1.0.0, los ADR, y el repositorio público con licencia MIT

Cuadro 3: Instanciación del modelo de proceso DSR de Peffers et al. [25].

Nota honesta sobre las iteraciones DSR: el modelo de Peffers es explícitamente iterativo (el resultado de la evaluación puede regresar a cualquier actividad anterior). Este proyecto sí iteró en la práctica — por ejemplo, la evaluación de seguridad de la Fase 5 (actividad 5) encontró un secreto JWT hardcodeado que obligó a regresar a la actividad 3 (rediseño de la configuración de seguridad) — pero esas iteraciones no se planificaron como tales desde el inicio del proyecto; se documentan aquí de forma retrospectiva, con la fecha y el hallazgo real que las disparó, no como un proceso DSR diseñado formalmente desde la Entrega 1A.

5.2. Esquema Goal-Question-Metric

Se instancia un esquema GQM [4] completo para la meta de rendimiento bajo carga, la única dimensión de la evaluación empírica con datos suficientes para un análisis estadístico inferencial real (Tabla 4).

Nivel	Contenido
Meta (Goal)	Analizar el sistema de caché Redis con el propósito de caracterizar su efecto sobre la latencia, respecto al tiempo de respuesta del endpoint <code>/api/v1/universidades</code> , desde el punto de vista del equipo de desarrollo, en el contexto de carga moderada (50 usuarios concurrentes)
Pregunta 1 (Question)	¿Cuál es la latencia típica (mediana, percentiles) del endpoint en estado de caché fría vs. caliente?
Métrica 1.1	Percentiles p50/p90/p95/p99 de latencia (ms), $n = 30$ por escenario
Métrica 1.2	Media y desviación estándar, con intervalo de confianza del 95 %
Pregunta 2 (Question)	¿La diferencia observada entre caché fría y caliente es estadísticamente significativa, y de qué magnitud práctica?
Métrica 2.1	Estadístico U y valor p del test de Mann-Whitney/Wilcoxon (dos muestras independientes)
Métrica 2.2	Tamaño de efecto (correlación biserial de rangos)
Pregunta 3 (Question)	¿El sistema cumple el requisito no funcional RNF-01 (respuesta < 500ms bajo 50 usuarios concurrentes)?
Métrica 3.1	p95 de <code>http_req_duration</code> en cada una de las 5 corridas k6, comparado contra el umbral de 500 ms

Cuadro 4: Esquema GQM para la meta de rendimiento del sistema de caché.

Los resultados de este esquema GQM se presentan en la Sección 8 y se reproducen ejecutando `scripts/perf-analysis.ipynb` contra los datos crudos en `k6/`.

5.3. Instrumentos y herramientas

- **Carga:** k6 v2.2.0 (`k6/load-test.js`)
- **Calidad web:** Lighthouse (vía `npx lighthouse`), contra build de producción real
- **Seguridad estática:** SpotBugs 4.8.6.4 + find-sec-bugs 1.13.0
- **Seguridad dinámica:** OWASP ZAP baseline scan
- **Cobertura:** JaCoCo 0.8.11
- **Análisis estadístico:** Python 3.14, `scipy.stats` (Mann-Whitney U), `numpy`
- **Entorno:** versiones exactas documentadas en `docs/entorno/versions.txt`

6. Diseño del Sistema y Arquitectura

6.1. Modelo C4

La arquitectura se documenta en tres niveles del modelo C4 (`docs/arquitectura/README.md`):

- **Nivel 1 (Contexto):** el sistema se relaciona con cuatro tipos de actores (Estudiante, Docente/Jurado, Coordinador, Administrador) y un sistema externo (API de universidades de Hipo Labs, consumida vía `ExternalApiServiceImpl` con caché Redis de 10 minutos).
- **Nivel 2 (Contenedores):** cuatro contenedores desplegados — Frontend Angular (SPA), Backend Spring Boot (API REST), PostgreSQL 15 (persistencia), Redis 7 (caché), coordinados por nginx como proxy inverso.
- **Nivel 3 (Componentes):** 25 controladores REST, organizados por dominio (Auth, Solicitudes, Jurados, Cronograma, Evaluaciones, Actas, Notificaciones, Reportes, Salas, Usuarios, Anteproyectos, y otros de soporte), cada uno delegando a una capa de servicio y, para la lógica académica compleja, a procedimientos almacenados PL/pgSQL.

6.2. Resumen de los siete Registros de Decisión Arquitectónica (ADR)

ADR	Decisión	Justificación resumida
ADR-001	Arquitectura general de 3 capas	Separación Angular/Spring Boot/PostgreSQL para independencia de despliegue y escalado
ADR-002	Autenticación JWT + cookies HTTP-Only	Stateless para escalar horizontalmente sin sesión compartida; cookie HTTP-Only mitiga XSS sobre el token
ADR-003	Estrategia híbrida JPA + procedimientos almacenados	CRUD simple vía Spring Data JPA; lógica académica compleja (notas ponderadas, reportes, firmas) vía PL/pgSQL para evitar SQL dinámico. Ampliada en la Fase 8 con la decisión explícita de mantener la convención Flyway en vez de espejar en <code>db/procs/</code>
ADR-004	Frontend Angular (SPA)	Reactividad y componentización para un dominio con múltiples roles y vistas condicionales
ADR-005	Seguridad alineada a OWASP	JWT de 7 claims, rate limiting, cabeceras HSTS/CSP/X-Frame-Options, BCrypt
ADR-006	Despliegue con Docker Compose, imágenes ancladas por digest SHA-256	Reproducibilidad determinista. Ampliada en la Fase 8 con la decisión de Railway como proveedor de producción (Sección 7)
ADR-007	Control de acceso por permisos dinámicos (BD), no roles estáticos en código	Reemplaza el RBAC fijo con <code>@PreAuthorize</code> descrito originalmente en ADR-005 por una verificación contra <code>permisos/rol_permisos</code> , para que cambiar qué rol puede qué requiera solo datos, no redespiegue

Cuadro 5: Resumen de los 7 ADR reales del repositorio (`docs/adr/`).

Corrección de integridad relevante: ADR-006 afirmaba, en su redacción original, que las imágenes base ya estaban ancladas por digest SHA-256 en `docker-compose.yml`. Verificado contra el archivo real durante la Fase 8 de este proyecto: esa afirmación era falsa (las imágenes usaban solo tags mutables). Se implementó el anclaje real en esa misma revisión, con los tres digests verificados vía `docker pull`, en vez de simplemente corregir el texto de la ADR sin corregir el código que describía.

6.3. Modelo de datos

El esquema relacional (`presus`, PostgreSQL 15) modela 13 entidades principales (usuarios, solicitudes, anteproyectos, jurados, cronogramas, evaluaciones, actas, rúbricas, salas, notificaciones, entre otras) con claves foráneas que garantizan integridad referencial. Los identificadores son `BIGINT` auto-

incrementales (migrados desde UUID en una corrección temprana documentada en OBS-01), y cuatro procedimientos almacenados PL/pgSQL puros (`sp_calcular_promedio_evaluacion`, `sp_asignar_jurado_mas`, `sp_firmar_acta_digital`, `sp_generar_reporte_defensas`) encapsulan la lógica académica que involucra uniones multi-tabla, versionados vía Flyway en `backend/src/main/resources/db/migration/`.

7. Implementación

7.1. Stack tecnológico

Capa	Tecnología y versión
Frontend	Angular 21.2.12, servido en producción por nginx (build multi-stage, ver <code>Frontend/Dockerfile</code>)
Backend	Spring Boot 3.2.1 sobre Java 17 (compilado con JDK 21 en modo <code>-release 17</code>)
Persistencia	PostgreSQL 15 (Flyway para migraciones versionadas)
Caché	Redis 7, vía <code>spring-boot-starter-data-redis</code>
Autenticación	JJWT 0.12.5, JWT de 7 claims RFC 7519, refresh token con blacklist en Redis
Documentación API	Springdoc OpenAPI 2.3.0 (Swagger UI), versionado dinámico <code>/api/v1/</code>
Generación de PDF	iText (actas y reportes)
CI/CD	GitHub Actions (<code>.github/workflows/ci.yml</code> , <code>backup.yml</code>)

Cuadro 6: Stack tecnológico real del sistema.

7.2. Seguridad implementada

La autenticación usa JWT con los 7 claims de RFC 7519 (`jti`, `iss`, `sub`, `aud`, `iat`, `nbf`, `exp`), refresh token con rotación y blacklist almacenados en Redis, rate limiting en `/api/v1/auth/login` (6 intentos/60s → HTTP 429), y cabeceras de seguridad HTTP (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) configuradas tanto en Spring Security como en nginx. El control de acceso usa `@PreAuthorize` por rol (ADMIN, DOCENTE, COORDINADOR, ESTUDIANTE). Los procedimientos almacenados y las consultas JPA son parametrizados; el análisis estático con find-sec-bugs confirmó 0 hallazgos de la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` (Sección 8).

7.3. Integración de API externa y caché

El servicio `ExternalApiServiceImpl` consume la API pública de universidades de Hipo Labs vía `WebClient` reactivo, con reintentos con backoff exponencial (3 intentos) ante fallos de red, y resultado cacheado en Redis (`@Cacheable`) con TTL de 10 minutos para universidades y 5 minutos para solicitudes. Este es el componente cuyo efecto de caché se mide cuantitativamente en la Sección 8 (RQ2).

7.4. Despliegue

El sistema se ejecuta localmente con `docker compose up -d --build` (cuatro contenedores: postgres, redis, backend, nginx+frontend). Para producción, se preparó — pero a la fecha de este informe

no se ha completado — un despliegue en Railway (`docs/despliegue/`), con un servicio Railway por contenedor, comunicados por red privada, y HTTPS gestionado automáticamente por el proveedor. La preparación incluyó: un **Frontend/Dockerfile** nuevo (antes no existía, el frontend solo se montaba como volumen local), la corrección de un **package-lock.json** desincronizado que rompía **npm ci** en un entorno Linux limpio, y la habilitación del detalle completo de `/actuator/health` (antes solo devolvía `{"status":"UP"}` sin desglose de componentes).

7.5. Usuario de demostración

Se sembró un usuario de demostración (`demo@uteq.edu.ec`, rol Coordinador) además del administrador, para que un evaluador externo pueda explorar el flujo académico completo sin necesidad de registrarse — credenciales publicadas en `README.md`.

8. Evaluación Empírica de Resultados

Esta sección presenta los resultados reales de cada bloque de medición. Cada cifra citada aquí es trazable a un archivo crudo y un comando de reproducción documentado en `docs/mediciones/DATA-PROVENANCE.md`.

8.1. Rendimiento

8.1.1. Carga bajo 50 usuarios concurrentes

Cinco corridas independientes válidas de k6 (`k6/load-test.js`, perfil de carga: rampa a 20 usuarios en 30s, sostenido en 50 usuarios por 1 minuto, rampa a 0; `run3-run7`) dan un p95 de `http_req_duration` entre 6.17 ms y 8.53 ms según la corrida (media 7.45 ms), muy por debajo del umbral RNF-01 de 500 ms. Dos corridas anteriores (`run1/run2`, metodología distinta con login repetido en cada iteración) se conservan como evidencia de una corrida inválida (100 % de fallos en su check principal) pero se excluyen de esta cifra — ver `k6/README.md`.

8.1.2. Efecto de la caché Redis (estadística descriptiva e inferencial)

Se midieron 30 muestras de latencia del endpoint `/api/v1/universidades` en estado de caché fría y 30 en caché caliente (`k6/cache-cold-samples.txt`, `cache-warm-samples.txt`).

Escenario	Media (ms)	DE	p50	p95	IC 95 %
Caché fría ($n = 30$)	109.62	18.88	106.02	108.07	[102.57, 116.67]
Caché caliente ($n = 30$)	7.86	0.75	7.60	8.89	[7.58, 8.14]

Cuadro 7: Estadística descriptiva de latencia, caché fría vs. caliente.

Prueba inferencial: Mann-Whitney U (equivalente a Wilcoxon rank-sum para dos muestras independientes, apropiado por la asimetría observada en caché fría) [1]: $U = 900$, $p = 3,02 \times 10^{-11}$ (dos colas), tamaño de efecto (correlación biserial de rangos) $r = 1,00$ (separación perfecta entre las dos distribuciones). La caché reduce la latencia media **13.9**× ($109.62 \text{ ms} \rightarrow 7.86 \text{ ms}$). Reproducible con `scripts/perf-analysis.ipynb`.

8.2. Seguridad

- **Revisión manual OWASP Top 10:** 6 de 10 controles auditados manualmente en código fuente; 3 hallazgos reales corregidos, 1 abierto (`docs/mediciones/sec/owasp/OWASP-AUDIT.md`).
- **OWASP ZAP (escaneo dinámico):** 0 hallazgos FAIL en las tres corridas reales (17-08 y 29-08). 5 hallazgos corregidos en total: 4 cabeceras de seguridad faltantes en nginx (17-08) y 1 alerta de riesgo High — `@angular/core` 21.2.12 vulnerable (3 CVEs reales), corregida el 29-08 actualizando a 21.2.22 — verificado contra la topología real de producción. 3 WARN de severidad Medium/Informational quedan abiertos.

- **Análisis estático (find-sec-bugs, 233 hallazgos totales al 29-08, 189 al 17-08 — el aumento refleja código de producción nuevo, no nuevas categorías de riesgo):** 160 informativos (SPRING_ENDPOINT), 52 CRLF_INJECTION_LOGS, 11 IMPROPER_UNICODE, 9 PATH_TRAVERSAL_IN, 0 UNSAFE_HASH_EQUALS (corregido en Fase 3, confirmado que se mantiene en 0), 1 informativo SPRING_CSRF_PROTECTION_DISABLED. **0 hallazgos de SQL_NONCONSTANT_STRING_PASSED** — ninguna evidencia de SQL dinámico/inyección.
- **npm audit (dependencias frontend):** bajó de 41 a **14 vulnerabilidades** (0 bajas, 4 moderadas, 5 altas, 5 críticas) tras corregir @angular/core el 29-08; las 14 restantes están en herramientas de build de íconos (sharp/to-ico/jimp), no en código que se envía al navegador — declaradas como brecha abierta (Sección 11).

8.3. Usabilidad

Sin datos reales. El instrumento SUS [7] está listo (docs/mediciones/sus/SUS-RESULTS.md), pero no se ha aplicado a participantes reales. Una versión anterior de este proyecto afirmaba un puntaje de 91.25/100 con 10 evaluadores — esos datos eran fabricados y fueron retirados explícitamente. No se reporta ninguna cifra de usabilidad en este informe porque no existe ninguna medición real que reportar.

8.4. Cobertura de código

JaCoCo sobre 109 pruebas automatizadas reales: 33.18 % de instrucciones (5,388/16,237), 37.06 % de líneas (1,118/3,017), 23.06 % de ramas (255/1,106) — por debajo del umbral RNF-04 de 60 %. El porcentaje había bajado respecto a una medición intermedia del 17-08 (23.33 %/28.45 %) porque se agregó código de producción real sin tests proporcionales en ese periodo; se recuperó y superó agregando 48 tests reales nuevos (2026-08-29) para 5 clases de servicio con lógica de negocio real y 0 % de cobertura previa, incluyendo las tres que invocan procedimientos almacenados sin prueba dedicada (sp_calcular_promedio_evaluacion, sp_generar_reporte_defensas, sp_firmar_acta_digital vía ActaServiceImpl.generarActa() — ver Sección 6) (docs/mediciones/jacoco/COVERAGE.md).

8.5. Calidad web (Lighthouse)

Seis corridas contra el build de producción real (3 desktop + 3 mobile, no contra el servidor de desarrollo, corrección metodológica de una medición anterior que sí usaba ng serve):

Perfil	Performance (media, $n = 3$)	Accesibilidad	Buenas Prácticas	SEO
Desktop	65	100	100	91
Mobile	61	100	100	91

Cuadro 8: Lighthouse, media de 3 corridas por perfil (2026-08-30; Accesibilidad subió de 89 a 100 tras corregir contraste de color y un landmark <main> faltante).

Con $n = 3$ por perfil, no se reporta una prueba de hipótesis formal entre desktop y mobile — el tamaño de muestra es insuficiente para una inferencia estadística significativa; se reporta solo la diferencia descriptiva (4 puntos). Performance sigue bajo el umbral de 80/100 en ambos perfiles. El Total Blocking Time (0 ms), el Cumulative Layout Shift (~ 0.01 – 0.03), el tiempo de respuesta del servidor y el trabajo de hilo principal ya son ideales (score 1 en los 4 audits); el cuello de botella es First Contentful Paint. La hipótesis original (contención de CPU por aplicaciones de escritorio activas) se puso a prueba de verdad el 2026-08-30: se cerraron Discord/Brave/Epic Games Launcher (confirmados corriendo con `Get-Process`) y se repitió la medición — el promedio de Performance prácticamente no cambió, refutando esa hipótesis en vez de solo repetirla. La causa real del FCP elevado queda declarada como no identificada, en vez de atribuida a una explicación no verificada — ver la nota metodológica completa en `docs/mediciones/perf/lighthouse/LIGHTHOUSE-REPORT.md` y la discusión de esta limitación en Sección [10](#).

9. Discusión

RQ1: ¿Puede automatizarse el flujo completo con trazabilidad verificable?

Sí, parcialmente verificado. Los 12 requisitos funcionales están implementados y el flujo completo (solicitud → anteproyecto → jurado → cronograma → evaluación → acta) se verificó manualmente end-to-end contra la topología de producción. Sin embargo, la trazabilidad **automatizada** solo cubre 5 de 12 requisitos con prueba dedicada (Sección 4) — la respuesta a RQ1 es “sí, funcionalmente” pero “parcialmente, en términos de verificación automatizada continua”.

RQ2: ¿Cuál es el efecto cuantificable de la caché sobre la latencia?

Respondida con evidencia estadística fuerte: reducción de latencia de $13.9 \times (109.62 \text{ ms a } 7.86 \text{ ms})$, estadísticamente significativa ($p = 3,02 \times 10^{-11}$) con separación perfecta entre distribuciones (tamaño de efecto $r = 1,00$). Es la pregunta de investigación con la respuesta más sólida de las cuatro, porque es la única con un diseño experimental que aísla una sola variable (estado de la caché) manteniendo todo lo demás constante.

RQ3: ¿Qué vulnerabilidades reales existen y qué tan cerradas están?

Ninguna vulnerabilidad de inyección SQL (0 hallazgos de `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` en 233 hallazgos totales de find-sec-bugs al 29-08), 0 fallos `FAIL` y 0 alertas de riesgo `High` en el escaneo dinámico OWASP ZAP (la única `High` encontrada, `@angular/core` vulnerable, se corrigió el 2026-08-29). Sin embargo, existen 14 vulnerabilidades de dependencias de terceros sin corregir (5 críticas, todas en herramientas de build de íconos que no llegan a producción) y 9 hallazgos de `PATH_TRAVERSAL_IN` sin resolver (riesgo real bajo, porque las entradas son IDs `Long`, no cadenas arbitrarias, pero no verificado formalmente). La respuesta honesta es: las vulnerabilidades de **código propio y de código servido al navegador** más críticas (inyección SQL, comparación de hash insegura, librería Angular vulnerable) están cerradas y verificadas; las vulnerabilidades restantes están en **herramientas de desarrollo**, no en código que se ejecuta en producción.

RQ4: ¿Qué tan honesta puede mantenerse la documentación bajo presión de plazos?

Esta pregunta se responde con el propio historial del proyecto, no con una medición externa. Se detectaron y corrigieron, dentro de este mismo proceso: una encuesta SUS fabricada (91.25/100 con 10 evaluadores inexistentes), métricas Lighthouse citadas sin haber corrido la herramienta, 5 citas de archivos de test inexistentes en la matriz de trazabilidad, una afirmación falsa sobre anclaje de imágenes Docker por digest, y una afirmación falsa sobre custodia de formularios de consentimiento firmados. En cada caso, la corrección fue reemplazar el dato fabricado por una medición real o una declaración explícita de ausencia — nunca por otro dato fabricado más plausible. Esto sugiere que sí es posible mantener honestidad bajo presión de plazos, al costo de reportar cifras menos favorables

(37.1 % de cobertura en vez de un “>60 %” fabricado; SUS “pendiente” en vez de “91.25/100”), lo cual es precisamente el trade-off que esta guía exige explícitamente priorizar.

10. Amenazas a la Validez

10.1. Validez de constructo

El p95 de latencia de k6 se usa como proxy de “rendimiento percibido”, pero no se midió tiempo de renderizado en el navegador ni Web Vitals bajo carga real de usuario (solo Lighthouse en condición de reposo, sin carga concurrente). El puntaje SUS, cuando se aplique, medirá usabilidad percibida declarada, no eficiencia real de tarea (tiempo para completar un flujo).

10.2. Validez interna

Las mediciones de rendimiento y Lighthouse se ejecutaron en la máquina de desarrollo personal de un integrante del equipo. El First Contentful Paint medido (2.8–6.1 s) es alto para un bundle de ~100 KB transferido servido en localhost sin latencia de red real; los audits que sí miden trabajo real de la aplicación (`server-response-time`, `bootup-time`, `mainthread-work-breakdown`, Total Blocking Time) puntúan perfecto, así que la causa no está en el código de la aplicación. La hipótesis inicial (2026-08-17) fue contención de CPU por aplicaciones de escritorio activas (Discord, Steam, Brave), confirmadas corriendo con `Get-Process | Sort CPU`. **Esa hipótesis se puso a prueba el 2026-08-30** en vez de repetirse sin verificar: se cerraron esas aplicaciones y se remidió — el promedio de Performance no cambió de forma apreciable (65/61 vs. 64/61). La amenaza a la validez interna sigue siendo real (algo en el entorno de medición local infla FCP/LCP de forma no representativa de un despliegue real), pero la causa concreta ya no puede afirmarse como “aplicaciones de escritorio” con la evidencia disponible; queda declarada como no identificada.

10.3. Validez externa

Los 50 usuarios concurrentes de k6 son un valor fijado por el RNF-01, no derivado de datos reales de uso institucional de la UTEQ (que no existen para este sistema, al ser nuevo). No hay garantía de que ese sea el nivel de concurrencia real durante un periodo de matrícula/defensas masivas. El usuario de demostración y las pruebas manuales no reemplazan una prueba piloto con usuarios reales del dominio (estudiantes, coordinadores) — que es precisamente la brecha que el SUS pendiente busca cerrar.

10.4. Validez de conclusión

El tamaño de muestra de Lighthouse ($n = 3$ por perfil) es demasiado pequeño para una prueba de hipótesis formal entre desktop y mobile (Sección 8); solo se reportan diferencias descriptivas. La revisión de trabajos relacionados (Sección 3) no siguió un protocolo de búsqueda sistemática formal con doble revisor, lo que limita la conclusión de que la cobertura de literatura es exhaustiva — se declaró explícitamente como una búsqueda dirigida, no una SLR completa. Finalmente, el propio historial de datos fabricados detectados en este proyecto (Sección 9, RQ4) es una amenaza de conclusión de segundo orden: obliga a que cada cifra en este informe sea trazable a

`docs/mediciones/DATA-PROVENANCE.md`, precisamente porque la confianza por defecto en la documentación de este proyecto específico ya se demostró insuficiente en al menos cinco episodios distintos.

11. Trabajo Futuro

Se declara explícitamente lo que quedó fuera de alcance de este informe, en vez de omitirlo o maquillarlo con contenido de relleno:

1. **Despliegue público en producción:** preparado completamente (Dockerfiles verificados, configuración de Railway, documentación de despliegue), pero la cuenta y el despliegue real requieren una acción del equipo humano que no se completó a la fecha de este informe.
2. **Cobertura de pruebas automatizadas:** 1 de 12 requisitos funcionales (RF-06, generación de actas) no tiene prueba automatizada dedicada (RF-02 y RF-04 se cubrieron el 2026-08-29 con `SolicitudServiceImplTest/CronogramaServiceImplTest`). Cobertura general de 37.06 % de líneas (JaCoCo, 2026-08-29, 109 tests), contra un objetivo de 60 %.
3. **Aplicación real del instrumento SUS:** el cuestionario está listo; aplicarlo a un grupo real de al menos 5–10 usuarios (estudiantes, docentes, coordinadores) es la tarea pendiente más directa y de menor esfuerzo entre las listadas aquí.
4. **Corrección de las 14 vulnerabilidades de dependencias de build restantes en el frontend** (5 críticas, en `sharp/to-ico/jimp`, herramientas de generación de íconos que no se envían al navegador) — la vulnerabilidad crítica que sí afectaba código servido a producción (`@angular/core`, detectada por ZAP) se corrigió el 2026-08-29.
5. **Revisión sistemática de literatura formal:** la Sección 3 es una búsqueda dirigida, no una SLR completa con protocolo pre-registrado y doble revisor independiente, siguiendo Kitchenham y Charters [17].
6. **Persistencia de archivos subidos en producción:** el filesystem del contenedor backend es efímero en el despliegue de Railway planeado; los PDF de anteproyectos y actas se perderían en cada redeploy sin un volumen persistente o almacenamiento de objetos externo.
7. **Firma real del docente-director sobre el SRS v1.0.0:** la página de aprobación está lista; la firma en sí es una acción pendiente del docente, no del equipo de desarrollo.
8. **Rediseño de la rúbrica de cobertura:** priorizar pruebas de integración reales (`@SpringBootTest` con base de datos real) sobre pruebas unitarias adicionales, dado que actualmente el proyecto no tiene ninguna prueba de integración verdadera (solo unitarias y slice tests `@WebMvcTest`).

Esta lista se prioriza deliberadamente: los ítems 1–3 son alcanzables en días con el equipo humano actuando (no requieren investigación adicional, solo ejecución de lo ya preparado); los ítems 4–8 requieren más tiempo de desarrollo o coordinación externa (el docente-director, en el caso del ítem 7).

12. Conclusiones

Este trabajo presentó el diseño, implementación y evaluación empírica del Sistema de Gestión de Pre-Sustentaciones UTEQ, siguiendo Design Science Research [25] de forma instanciada y verificable. Los 12 requisitos funcionales están implementados y el flujo académico completo se verificó funcionando end-to-end contra una topología de producción real. La evaluación empírica combina estadística descriptiva e inferencial genuina — el hallazgo más sólido es la reducción de latencia de $13.9\times$ por efecto de caché Redis, con significancia estadística fuerte ($p < 10^{-10}$) y tamaño de efecto máximo — y auditoría de seguridad multi-herramienta que confirma ausencia de inyección SQL en el código propio; se corrigió la única vulnerabilidad de dependencias que afectaba código servido al navegador (@angular/core, detectada por ZAP), y quedan abiertas 14 vulnerabilidades en herramientas de build que no se envían a producción.

La contribución más distintiva de este informe no es técnica sino metodológica: el proceso de elaboración de este mismo proyecto incluyó la detección real de datos académicos fabricados en fases anteriores (usabilidad, rendimiento web, evidencia de trazabilidad, integridad de configuración de despliegue, custodia de consentimientos informados) y su corrección sistemática, documentada de forma trazable en `docs/mediciones/DATA-PROVENANCE.md` y en la bitácora de cada fase del proyecto. Esa corrección no fue cosmética: implicó reportar una cobertura de código de 37.1 % en vez de un “>60 %” previamente afirmado sin evidencia, y declarar la usabilidad como “sin datos” en vez de mantener un 91.25/100 fabricado.

Las brechas que quedan (cobertura de pruebas incompleta, ausencia de datos SUS reales, despliegue público pendiente, dependencias vulnerables sin corregir) se declaran explícitamente en la Sección 11 en vez de ocultarse o rellenarse con contenido que simule completitud. Esta decisión — priorizar un informe más corto pero verificable sobre uno más extenso pero con contenido fabricado — es, en sí misma, la conclusión metodológica central de este trabajo: en un proyecto con antecedentes reales de fabricación de evidencia, la integridad de lo que se reporta importa más que la completitud aparente de lo que se reporta.

Declaraciones

Disponibilidad de código

El código fuente completo está disponible públicamente bajo licencia MIT en <https://github.com/carla22072004/PFC-Presustentaciones-2026>.

Disponibilidad de datos

Los datos crudos de todas las mediciones empíricas citadas en este informe (corridas k6, reportes Lighthouse, reportes OWASP ZAP, reporte find-sec-bugs, reporte JaCoCo) están versionados en el mismo repositorio, con su procedencia documentada en `docs/mediciones/DATA-PROVENANCE.md`. No se generó ningún dato de participantes humanos (la encuesta SUS no se ha aplicado todavía — ver Sección 8), por lo que no aplica una declaración de anonimización de datos de sujetos humanos en este informe.

Disponibilidad de materiales

Los scripts de análisis (`scripts/gen-figuras.py`, `scripts/perf-analysis.ipynb`, `scripts/sus-analysis.ipynb`, `scripts/validate-traceability.sh`, `scripts/audit-sql-dynamic.sh`), el script de carga k6, y los archivos de configuración de despliegue están en el mismo repositorio bajo la misma licencia.

Declaración ética

Este proyecto no involucró experimentación con seres humanos con datos reales a la fecha de este informe (el instrumento SUS existe pero no se ha aplicado). El protocolo de consentimiento informado para cuando se aplique está documentado en `docs/etica/consentimientos/plantilla-consentimiento.md`. Se declara explícitamente, como parte de esta misma sección, que una versión anterior del documento ético del proyecto (`docs/etica/ETHICS.md`) afirmaba falsamente que existían 10 formularios de consentimiento firmados en custodia física para una encuesta SUS que en realidad nunca se aplicó; esa afirmación fue detectada y corregida durante este proceso, y se documenta aquí por transparencia.

Taxonomía CRediT (Contributor Roles Taxonomy)

Se aplican los 14 roles estándar de CRediT [6] a los cuatro integrantes del equipo. La Tabla 9 reemplaza la versión anterior de `CONTRIBUTORS.md`, que mezclaba roles CRediT reales con descripciones de proyecto no estandarizados (p. ej. “UI/UX Design”, “DevOps (Docker)”); esta tabla usa exclusivamente los 14 términos oficiales de la taxonomía.

Rol CRediT	Alava	Moncayo	Zamora	Barreto
Conceptualization	✓			
Data curation			✓	
Formal analysis	✓			✓
Funding acquisition	—	—	—	—
Investigation	✓	✓	✓	✓
Methodology	✓		✓	
Project administration	✓			
Resources				✓
Software	✓	✓	✓	✓
Supervision	—	—	—	—
Validation			✓	✓
Visualization		✓		
Writing – original draft			✓	
Writing – review & editing	✓	✓	✓	✓

Cuadro 9: Asignación de los 14 roles CRediT. “—” indica un rol que genuinamente no aplica a ningún integrante en este proyecto (no hubo financiamiento externo ni un supervisor formal más allá del docente-director, cuyo rol se declara por separado, no como coautor).

Nota: Alava Alvarado concentró la seguridad backend (JWT/Spring Security) y la administración del proyecto; Moncayo Loor, el frontend Angular; Zamora Arias, el diseño de base de datos y la documentación técnica; Barreto Rosado, las pruebas y la infraestructura Docker. Los cuatro participaron en revisión de código y redacción/edición de la documentación (fila “Writing – review & editing”), consistente con lo ya declarado en `CONTRIBUTORS.md`.

Conflictos de interés

Los autores declaran no tener conflictos de interés financieros ni personales que pudieran haber influido en este trabajo.

Financiamiento

Este proyecto no recibió financiamiento externo. Se desarrolló como parte de las actividades académicas de la asignatura Aplicaciones Web, Quinto Nivel, Carrera de Ingeniería de Software, UTEQ.

Declaración de uso de asistentes de inteligencia artificial

Herramienta	Versión/modelo	Propósito y sección donde se usó
Claude (Anthropic), vía Claude Code	Claude Sonnet 5	Implementación de código (backend, frontend, infraestructura), redacción y corrección de documentación técnica, ejecución y análisis de pruebas empíricas (k6, Lighthouse, JaCoCo, OWASP ZAP, find-sec-bugs), búsqueda y verificación de referencias bibliográficas, y redacción de este informe final completo (todas las secciones). Detalle completo en <code>docs/etica/ai-disclosure.md</code> .

Cuadro 10: Declaración de uso de IA generativa.

Se declara explícitamente que ninguna cifra empírica de este informe fue generada por el asistente de IA sin ejecutar realmente la herramienta correspondiente contra el sistema real — ver la política completa en `docs/etica/ai-disclosure.md`, que documenta también los episodios detectados de datos fabricados en fases anteriores del proyecto (no atribuibles a este asistente, dado que ocurrieron antes de su intervención documentada en el historial de commits) y cómo se corrigieron.

Agradecimientos

Los autores agradecen al Ing. Guerrero Ulloa Gleiston Cícero por la retroalimentación docente que motivó las correcciones documentadas en `docs/observaciones/OBSERVACIONES.md`, y al Equipo E (Tejada Bajaña Luis Alejandro, Alava Alvarado Jean Pierre) por la evaluación cruzada que identificó las brechas E1-E10 cerradas en la Fase 5 de este proyecto.

Referencias

Nota de integridad bibliográfica: las 32 referencias siguientes fueron buscadas y verificadas individualmente (autor, año, venue, DOI cuando disponible) mediante búsqueda web dirigida durante la elaboración de este informe, no generadas de memoria. **19 de las 32 (59.4 %)** cumplen el criterio estricto de alto impacto exigido por esta guía (revista Q1/Q2 indexada en Scopus, o actas de ICSE/FSE/ASE/MSR/EASE/ESEM) — marcadas con † abajo. Esto queda **1 referencia por debajo** del mínimo de 20 solicitado. Se declara esta brecha explícitamente: se decidió no forzar una referencia adicional de relevancia dudosa solo para alcanzar el número exacto, dado que las reglas transversales de esta guía penalizan más severamente una referencia fabricada o forzada que una cifra ligeramente por debajo del mínimo con la brecha declarada honestamente.

Referencias

- [1] † A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proc. 33rd Int. Conf. Software Engineering (ICSE)*, 2011, pp. 1–10. DOI: 10.1145/1985793.1985795.
- [2] † A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proc. 35th Int. Conf. Software Engineering (ICSE)*, 2013, pp. 712–721. DOI: 10.1109/ICSE.2013.6606617.
- [3] † A. Bangor, P. Kortum, and J. Miller, “An empirical evaluation of the System Usability Scale,” *International Journal of Human-Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008. DOI: 10.1080/10447310802205776.
- [4] V. R. Basili, G. Caldiera, and H. D. Rombach, “The Goal Question Metric approach,” in *Encyclopedia of Software Engineering*, vol. 1. Wiley, 1994, pp. 528–532.
- [5] † M. Beller, G. Gousios, and A. Zaidman, “Oops, my tests broke the build: An explorative analysis of Travis CI with GitHub,” in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 356–367. DOI: 10.1109/MSR.2017.62.
- [6] A. Brand, L. Allen, M. Altman, M. Hlava, and J. Scott, “Beyond authorship: attribution, contribution, collaboration, and credit,” *Learned Publishing*, vol. 28, no. 2, pp. 151–155, 2015. DOI: 10.1087/20150211.
- [7] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London: Taylor & Francis, 1996, pp. 189–194. DOI: 10.1201/9781498710411-35.
- [8] A. Cockburn, *Writing Effective Use Cases*. Boston: Addison-Wesley, 2001.
- [9] † A. Decan, T. Mens, and E. Constantinou, “On the impact of security vulnerabilities in the npm package dependency network,” in *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, 2018, pp. 181–191. DOI: 10.1145/3196398.3196401.

- [10] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” martin-fowler.com, 2014.
- [11] † K. Gallaba, M. Lamothe, and S. McIntosh, “Lessons from eight years of operational data from a continuous integration service,” in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1330–1342. DOI: 10.1145/3510003.3510211.
- [12] † A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [13] † M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, “Usage, costs, and benefits of continuous integration in open-source projects,” in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2016, pp. 426–437. DOI: 10.1145/2970276.2970358.
- [14] INCOSE, *Guide for Writing Requirements*, INCOSE-TP-2010-006-04, 2023.
- [15] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. ISO, 2018.
- [16] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015. DOI: 10.17487/RFC7519.
- [17] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” EBSE Technical Report EBSE-2007-01, Keele Univ. and Univ. of Durham, 2007.
- [18] B. Lu, X. Zhang, Z. Ling, Y. Zhang, and Z. Lin, “A measurement study of authentication rate-limiting mechanisms of modern websites,” in *Proc. 34th Annual Computer Security Applications Conf. (ACSAC)*, 2018, pp. 89–100. DOI: 10.1145/3274694.3274714.
- [19] † P. Mäder and A. Egyed, “Do developers benefit from requirements traceability when evolving and maintaining a software system?,” *Empirical Software Engineering*, vol. 20, no. 2, pp. 413–441, 2015. DOI: 10.1007/s10664-014-9314-z.
- [20] † E. M. Maximilien and L. Williams, “Assessing test-driven development at IBM,” in *Proc. 25th Int. Conf. Software Engineering (ICSE)*, 2003, pp. 564–569.
- [21] † S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, “An empirical study of the impact of modern code review practices on software quality,” *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146–2189, 2016. DOI: 10.1007/s10664-015-9381-9.
- [22] OWASP Foundation, “OWASP Top 10:2021,” 2021.
- [23] † M. J. Page *et al.*, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021. DOI: 10.1136/bmj.n71.

- [24] † I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, “Vulnerable open source dependencies: counting those that matter,” in *Proc. 12th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2018. DOI: 10.1145/3239235.3268920.
- [25] † K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–78, 2007. DOI: 10.2753/MIS0742-1222240302.
- [26] P. Ralph *et al.*, “Empirical standards for software engineering research,” arXiv:2010.03525, 2021.
- [27] † A. Rezaei Nasab, M. Shahin, S. A. Hoseyni Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, “An empirical study of security practices for microservices systems,” *Journal of Systems and Software*, vol. 192, 2022. DOI: 10.1016/j.jss.2022.111563.
- [28] † P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8.
- [29] † B. Vasilescu, Y. Yu, H. Wang, P. Devanbu, and V. Filkov, “Quality and productivity outcomes relating to continuous integration in GitHub,” in *Proc. 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 805–816. DOI: 10.1145/2786805.2786850.
- [30] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin: Springer, 2012.
- [31] † C. Wohlin, “Guidelines for snowballing in systematic literature studies and a replication in software engineering,” in *Proc. 18th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, 2014, pp. 1–10. DOI: 10.1145/2601248.2601268.
- [32] † F. Zampetti, C. Vassallo, S. Panichella, G. Canfora, H. Gall, and M. Di Penta, “An empirical characterization of bad practices in continuous integration,” *Empirical Software Engineering*, vol. 25, no. 2, pp. 1095–1135, 2020. DOI: 10.1007/s10664-019-09785-8.